

Model Training_Conversion

[Model Training_Conversion](#)

[1. Model Training](#)

[2. Model Conversion](#)

[References](#)

After completing the dataset annotation tutorial content, we can use the board to start training the model.

This tutorial only introduces CLI model training and conversion. You can refer to the official website for Python case modifications.

Due to performance reasons of the Raspberry Pi 5 board, model training takes a long time and requires patience.

1. Model Training

Use CLI commands to directly train the model: Copy the yolo26n.pt file to the configuration file directory, then open a terminal in the configuration file directory:

```
cd /home/pi/ultralytics26/data/yahboom_data/orange_data
```

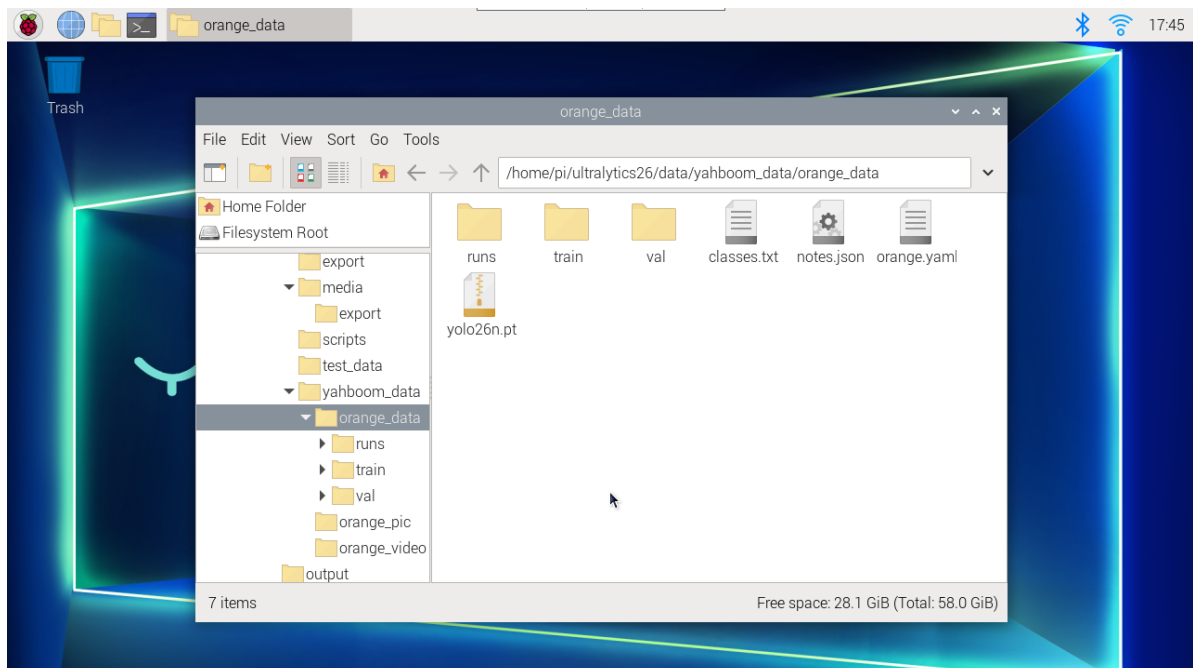
```
yolo detect train data=orange.yaml model=yolo26n.pt epochs=100 imgsz=640
```

data : Dataset configuration file

model : Pre-trained model file

epochs : Number of training epochs

imgsz : Specify input image size



```
pi@raspberrypi:~/ultralytics26/data/yahboom_data/orange_data $ yolo detect train data=orange.yaml model=yolo26n.pt epochs=100 imgsz=640
New https://pypi.org/project/ultralytics/8.4.21 available @ Update with 'pip install -U ultralytics'
ultralytics 8.4.19 Python-3.11.2 torch-2.1.2 CPU (Cortex-A76)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=RandAugment, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=orange.yaml, degrees=0.0, deterministic=True, device=cpu, dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None, end2end=None, epochs=100, erasing=0.4, exist_ok=False, flip_lr=0.5, flipud=0.0, format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.0, line_width=None, lr=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolo26n.pt, momentum=0.937, mosaic=1.0, multi_scale=0.0, name=train3, nbs=64, nms=False, opset=None, optimize=False, optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0, pretrained=True, project=None, project=None, rect=False, resume=False, retina_masks=False, rle=1.0, save=True, save_conf=False, save_crop=False, save_dir=/home/pi/ultralytics26/data/yahboom_data/orange_data/runs/detect/train3, save_frames=False, save_json=False, save_period=-1, save_txt=False, scales=0.5, seed=0, shear=0.0, show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True, single_cls=False, source=None, split=val, stream_buffer=False, task=detect, time=None, tracker=botsort.yaml, translate=0.1, val=True, verbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None
Overriding model.yaml nc=80 with nc=1

      from  n  params  module  arguments
0         -1  1      464  ultralytics.nn.modules.conv.Conv  [3, 16, 3, 2]
1         -1  1     4672  ultralytics.nn.modules.conv.Conv  [16, 32, 3, 2]
2         -1  1     6640  ultralytics.nn.modules.block.C3k2  [32, 64, 1, False, 0.25]
3         -1  1    36992  ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]
4         -1  1    26080  ultralytics.nn.modules.block.C3k2  [64, 128, 1, False, 0.25]
5         -1  1   147712  ultralytics.nn.modules.conv.Conv  [128, 128, 3, 2]
6         -1  1    87040  ultralytics.nn.modules.block.C3k2  [128, 128, 1, True]
7         -1  1   295424  ultralytics.nn.modules.conv.Conv  [128, 256, 3, 2]

train: Scanning /home/pi/ultralytics26/data/yahboom_data/orange_data/train/labels... 107 images, 0 backgrounds, 0 corrupt: 70% 107/153 143.7it/s
train: Scanning /home/pi/ultralytics26/data/yahboom_data/orange_data/train/labels... 127 images, 0 backgrounds, 0 corrupt: 83% 127/153 158.5it/s
train: Scanning /home/pi/ultralytics26/data/yahboom_data/orange_data/train/labels... 147 images, 0 backgrounds, 0 corrupt: 96% 147/153 164.2it/s
train: Scanning /home/pi/ultralytics26/data/yahboom_data/orange_data/train/labels... 153 images, 0 backgrounds, 0 corrupt: 100% 153/153 169.9it/s
0.9s
train: New cache created: /home/pi/ultralytics26/data/yahboom_data/orange_data/train/labels.cache
val: Fast Image access (ping: 0.0x0.0 ms, read: 80.8x2.1 MB/s, size: 409.7 KB)
val: Scanning /home/pi/ultralytics26/data/yahboom_data/orange_data/val/labels... 20 images, 0 backgrounds, 0 corrupt: 100% 20/20 200.6it/s 0.1s
val: New cache created: /home/pi/ultralytics26/data/yahboom_data/orange_data/val/labels.cache
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum' automatically...
optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 114 weight(decay=0.0), 126 bias(decay=0.0)
Plotting labels to /home/pi/ultralytics26/data/yahboom_data/orange_data/runs/detect/train3/labels.jpg...
Image sizes 640 train, 640 val
Using 0 dataloader workers
Logging results to /home/pi/ultralytics26/data/yahboom_data/orange_data/runs/detect/train3
Starting training for 100 epochs...

      Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
1/100      0G      0.4785      5.972      0.005731      18      640: 100% 10/10 32.4s/it 5:24
      Class  Images  Instances  Box(P  mAP50  mAP50-95): 100% 1/1 14.0s/it 14.0s
      all    20      20      0.00333      1      0.611      0.554

      Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
2/100      0G      0.5284      5.404      0.006319      21      640: 100% 10/10 31.5s/it 5:15
      Class  Images  Instances  Box(P  mAP50  mAP50-95): 100% 1/1 14.0s/it 14.0s
      all    20      20      0.00333      1      0.594      0.555

      Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
3/100      0G      0.5282      4.723      0.005789      18      640: 100% 10/10 31.9s/it 5:19
      Class  Images  Instances  Box(P  mAP50  mAP50-95): 100% 1/1 14.0s/it 14.0s
      all    20      20      0.00333      1      0.784      0.72
```

2. Model Conversion

The final model will be generated in the runs folder: Generally choose the best.pt file for use

```
/home/pi/ultralytics26/data/yahboom_data/orange_data/runs/detect/train/weights
```

Convert pt model to ncnn model:

```
cd
/home/pi/ultralytics26/data/yahboom_data/orange_data/runs/detect/train/weights
```

```
yolo export model=best.pt format=ncnn
```

```
pi@raspberrypi:~/ultralytics26/data/yahboom_data/orange_data/runs/detect/train3/weights $ ls
best.pt  last.pt
pi@raspberrypi:~/ultralytics26/data/yahboom_data/orange_data/runs/detect/train3/weights $ yolo export model=best.pt format=ncnn
ultralytics 8.4.19 Python-3.11.2 torch-2.1.2 CPU (Cortex-A76)
WARNING ⚠ NCNN export does not support end2end models, disabling end2end branch.
Yolo26n summary (fused): 146 layers, 2,494,694 parameters, 0 gradients, 5.2 GFLOPs

PyTorch: starting from 'best.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) (1, 5, 8400) (5.1 MB)

NCNN: starting export with NCNN 1.0.20241226 and PNNX 20260112...
pnnxparam = best_ncnn_model/model.pnnx.param
pnnxbtn = best_ncnn_model/model.pnnx.btn
pnnxpy = best_ncnn_model/model.pnnx.py
pnnxonnx = best_ncnn_model/model.pnnx.onnx
ncnnparam = best_ncnn_model/model.ncnn.param
ncnnbtn = best_ncnn_model/model.ncnn.btn
ncnnpy = best_ncnn_model/model.ncnn.py
fp16 = 0
optlevel = 2
device = cpu
inputshape = [1,3,640,640]f32
outputshape = [1,5,8400]f32
```

```

inputshape = [1,3,640,640]f32
##### pass_level0
inline module = torch.nn.modules.linear.Identity
inline module = ultralytics.nn.modules.block.Attention
inline module = ultralytics.nn.modules.block.Bottleneck
inline module = ultralytics.nn.modules.block.C2PSA
inline module = ultralytics.nn.modules.block.C3k
inline module = ultralytics.nn.modules.block.C3k2
inline module = ultralytics.nn.modules.block.PSABlock
inline module = ultralytics.nn.modules.block.SPPF
inline module = ultralytics.nn.modules.conv.Concat
inline module = ultralytics.nn.modules.conv.Conv
inline module = ultralytics.nn.modules.conv.DWConv
inline module = ultralytics.nn.modules.head.Detect
inline module = torch.nn.modules.linear.Identity
inline module = ultralytics.nn.modules.block.Attention
inline module = ultralytics.nn.modules.block.Bottleneck
inline module = ultralytics.nn.modules.block.C2PSA
inline module = ultralytics.nn.modules.block.C3k
inline module = ultralytics.nn.modules.block.C3k2
inline module = ultralytics.nn.modules.block.PSABlock
inline module = ultralytics.nn.modules.block.SPPF
inline module = ultralytics.nn.modules.conv.Concat
inline module = ultralytics.nn.modules.conv.Conv
inline module = ultralytics.nn.modules.conv.DWConv
inline module = ultralytics.nn.modules.head.Detect

-----

##### pass_level1
##### pass_level2
##### pass_level3
##### pass_level4
##### pass_level5
##### pass_ncnn
NCNN: export success 8.0s, saved as 'best_ncnn_model' (9.3 MB)

Export complete (9.5s)
Results saved to /home/pi/ultralytics26/data/yahboom_data/orange_data/runs/detect/train3/weights
Predict:      yolo predict task=detect model=best_ncnn_model imgs=640
Validate:     yolo val task=detect model=best_ncnn_model imgs=640 data=orange.yaml
Visualize:    https://netron.app
? Learn more at https://docs.ultralytics.com/modes/export
pi@raspberrypi:~/ultralytics26/data/yahboom_data/orange_data/runs/detect/train3/weights $

```

References

[Model Training with Ultralytics YOLO - Ultralytics YOLO Documentation](#)